



Laboratorio di Programmazione

Lezione 10 – Ricorsione, Liste, Undefined Behaviours

Marco Anisetti (teoria)

Dipartimento di Informatica
marco.anisetti@unimi.it

Matteo Luperto (lab. turno A)

Dipartimento di Informatica
matteo.luperto@unimi.it

Nicola Bena (lab. turno B)

Dipartimento di Informatica
nicola.bena@unimi.it

Ricorsione

Usata quando la soluzione del problema è definita in termini di soluzioni su istanze più piccole del problema stesso

Soluzione definita in termini di:

- Caso base: la soluzione può essere computata direttamente
- Passo ricorsivo: la soluzione viene espressa sulla base della soluzione di una diversa istanza, più piccola, dello stesso problema

Ricorsione

```
int recursive_function(int a){  
    if (a == 1) {  
        return 1;  
    }  
    return a + recursive_function(a-1);  
}
```

Ricorsione

```
int recursive_function(int a){
```

```
    if (a == 1) {  
        return 1;  
    }
```

Caso base: la soluzione può essere calcolata direttamente

```
    return a + recursive_function(a-1);  
}
```

Passo ricorsivo: la soluzione viene calcolata sulla base di un'istanza più piccola del problema, cioè $a-1$

Ricorsione

```
int recursive_function(int a){
```

```
    if (a == 1) {  
        return 1;  
    }
```

Caso base: la soluzione può essere calcolata direttamente

```
    return a + recursive_function(a-1);  
}
```

Passo ricorsivo: la soluzione viene calcolata sulla base di un'istanza più piccola del problema, cioè $a-1$

Che cosa calcola questa funzione?

Ricorsione

```
int recursive_function(int a){
```

```
    if (a == 1) {  
        return 1;  
    }
```

Caso base: la soluzione può essere calcolata direttamente

```
    return a + recursive_function(a-1);  
}
```

Passo ricorsivo: la soluzione viene calcolata sulla base di un'istanza più piccola del problema, cioè $a-1$

Che cosa calcola questa funzione? Sommatoria $[1, a]$: $\text{sommatoria}(a) = a + \text{sommatoria}(a-1)$

Ricorsione

```
int sommatoria(int a){  
    if (a == 0) {  
        return 0;  
    }  
    return a + sommatoria(a-1);  
}
```

Supponiamo $a=3$ inizialmente

1. Record di attivazione **r1**: $a=3$, chiamo `sommatoria(3-1)`
2. Record di attivazione **r2**: $a=2$, chiamo `sommatoria(2-1)`
3. Record di attivazione **r3**: $a=1$, chiamo `sommatoria(1-1)`
4. Record di attivazione **r4**: $a=0$, restituisco a **r3** 0
5. Record di attivazione **r3**: restituisco a **r2** $1+0$, cioè 1
6. Record di attivazione **r2**: restituisco a **r1** $2+1$, cioè 3
7. Record di attivazione **r1**: restituisco al main $3+3$, cioè 6

Ricorsione

```
int recursive_function(int a){  
    if (a == 1) {  
        return 1;  
    }  
    return a + recursive_function(a);  
}
```


Ricorsione

```
> ./ricorsione_v3_infinita 10 (base)  
fish: Job 1, './ricorsione_v3_infinita 10' terminated by signal  
SIGSEGV (Address boundary error)
```

```
int recursive_function(int a){  
    if (a == 1) {  
        return 1;  
    }  
    return a + recursive_function(a);  
}
```

Il processo continua a creare nuovi record di attivazione fino a sfiorare lo stack.
A quel punto il processo viene terminato dal sistema operativo



Dobbiamo assicurarci di computare la soluzione su un'istanza più piccola, altrimenti non arriverò mai al caso base e la ricorsione non finirà mai!

In Pratica

- Risolvere un problema ricorsivo con un algoritmo ricorsivo è la soluzione più naturale
 - Codice compatto, facile da leggere, *elegante*
- Ma...

In Pratica

- Risolvere un problema ricorsivo con un algoritmo ricorsivo è la soluzione più naturale
 - Codice compatto, facile da leggere, *elegante*
- Ma...
 - Ogni chiamata di funzione crea un nuovo record di attivazione sullo stack
 - Se il caso base non viene mai raggiunto, lo spazio sullo stack viene saturato
 - Debug non immediato, occorre fare attenzione all'ordine con cui le funzioni terminano
 - Trasformazioni da algoritmo ricorsivo a algoritmo iterativo (ad es., *tail recursion*, vedrete come fare in corsi più avanzati)

Ricorsione vs Iterazione

```
long sommatoria(long a){  
    if (a == 1) {  
        return 1;  
    }  
    return a + sommatoria(a-1);  
}
```

```
long sommatoria(long a){  
    long sum = 0, i;  
    for(i=1; i<=a; i++) {  
        sum += i;  
    }  
    return sum;  
}
```

Su input piccoli il tempo di esecuzione non ha (grosse) differenze...

Ricorsione vs Iterazione

```
long sommatoria(long a){
```

```
> time ./ricorsione_v4_long 1000000000 (base)
```

Executed in	419.92 millis	fish	external
usr time	2.79 millis	0.42 millis	2.37 millis
sys time	6.63 millis	2.37 millis	4.26 millis

fish: Job 1, 'time ./ricorsione_v4_long 10000...' terminated by signal SIGSEGV (Address boundary error)

```
}
```

La dimensione dello stack è grande, ma finita!



```
long sommatoria(long a){  
    long sum = 0, i;  
    for(i=1; i<=a; i++) {  
        sum += i;  
    }  
}
```

```
> time ./sommatoria 1000000000 (base)  
500000000500000000
```

Executed in	959.28 millis	fish	external
usr time	951.84 millis	0.28 millis	951.56 millis
sys time	6.62 millis	2.08 millis	4.54 millis

Su input piccoli il tempo di esecuzione non ha differenze... Ma su input grandi la ricorsione può essere un problema

Liste Concatenate

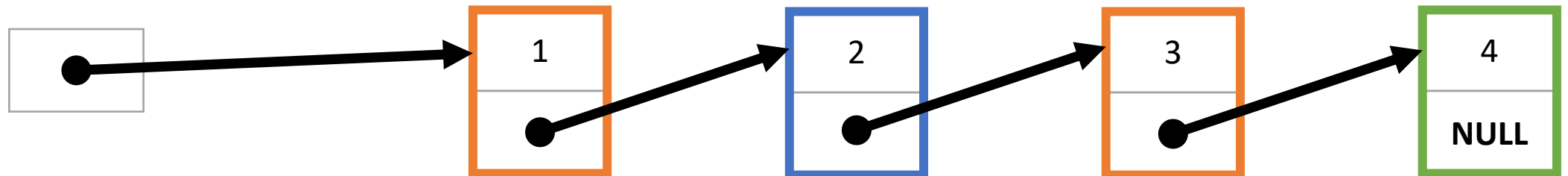
- Struttura dati per gestire sequenze dinamiche di elementi
- Ogni *nodo* della lista contiene l'elemento da memorizzare e un puntatore all'elemento successivo nella lista
- La lista non ha dimensione fissa: cambia sulla base del numero di elementi da memorizzare
- Uso efficiente della memoria: non serve allocare un array grande a sufficienza
- Costo da pagare: accedere ai dati è più costoso, ma si possono usare formati diversi più adatti al tipo di lettura (come: in corsi futuri)

Liste Concatenate

```
#include <stdio.h>
#include <stdlib.h>

// Struttura che rappresenta lista lineare concatenata
typedef struct Node {
    int data; // Campo data
    struct Node* next; // Puntatore al prossimo elemento
} Node;

void main(){
    // Testa della lista, nel main
    Node* head = NULL;
}
```

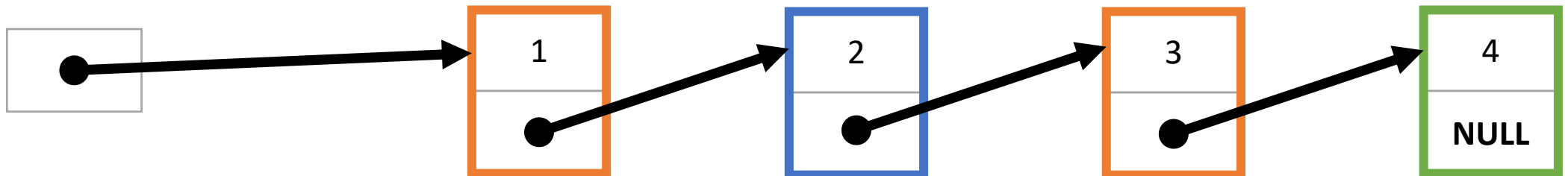


Liste Concatenate

```
#include <stdio.h>
#include <stdlib.h>
```

```
// Struttura che rappresenta lista lineare concatenata
typedef struct Node {
    int data; // Campo data
    struct Node* next; // Puntatore al prossimo elemento
} Node;
```

```
void main(){
// Testa della lista, nel main
Node* head = NULL;
}
```



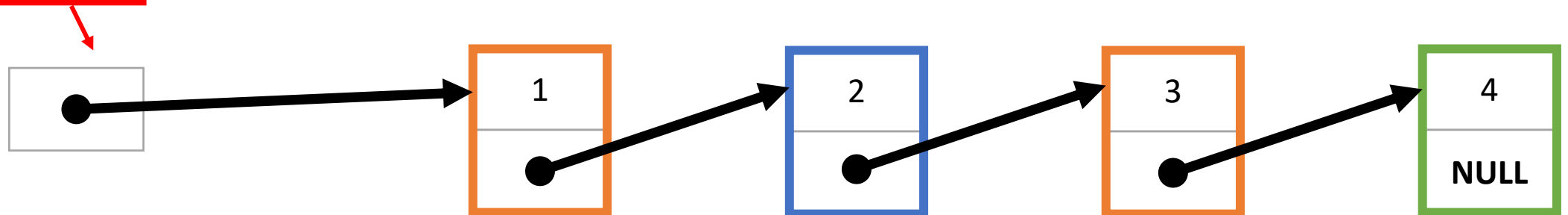
Struttura ricorsiva: è possibile implementarla perché il secondo campo è un puntatore (quindi di dimensione nota)
→ il compilatore conosce la dimensione di entrambi i campi (`sizeof(int)` e `sizeof(Node*)`)
→ Lo spazio occupato da una variabile di tipo puntatore è sempre lo stesso! (8 byte per sistemi a 64 bit)

Liste Concatenate

```
#include <stdio.h>
#include <stdlib.h>

// Struttura che rappresenta lista lineare concatenata
typedef struct Node {
    int data; // Campo data
    struct Node* next; // Puntatore al prossimo elemento
} Node;

void main(){
    // Testa della lista, nel main
    Node* head = NULL;
}
```



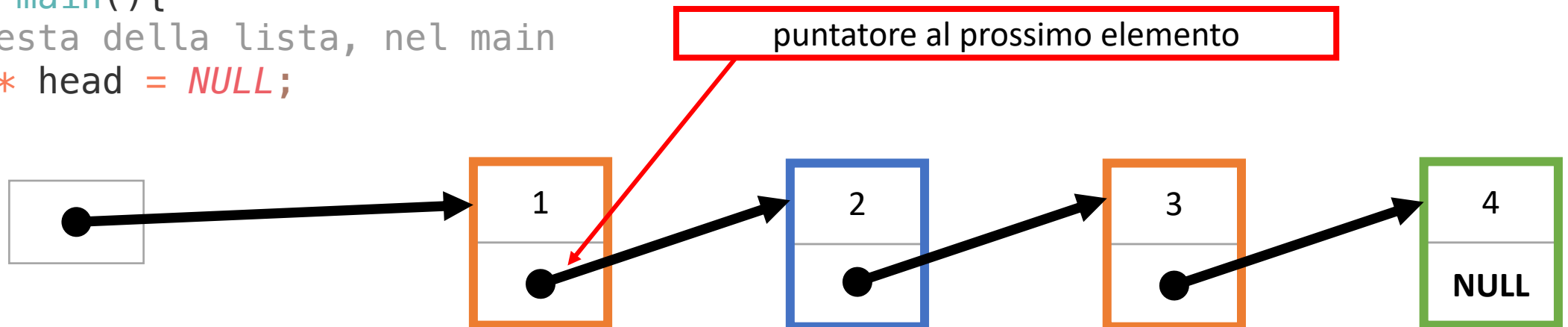
Liste Concatenate

```
#include <stdio.h>
#include <stdlib.h>
```

```
// Struttura che rappresenta lista lineare concatenata
typedef struct Node {
    int data; // Campo data
    struct Node* next; // Puntatore al prossimo elemento
} Node;
```

```
void main(){
// Testa della lista, nel main
Node* head = NULL;

}
```

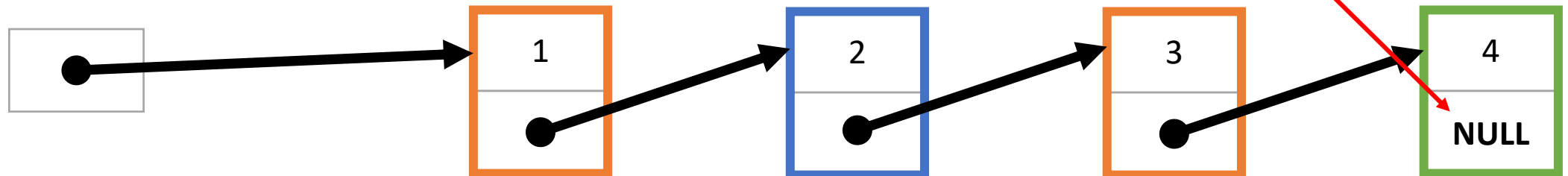


Liste Concatenate

```
#include <stdio.h>
#include <stdlib.h>

// Struttura che rappresenta lista lineare concatenata
typedef struct Node {
    int data; // Campo data
    struct Node* next; // Puntatore al prossimo elemento
} Node;

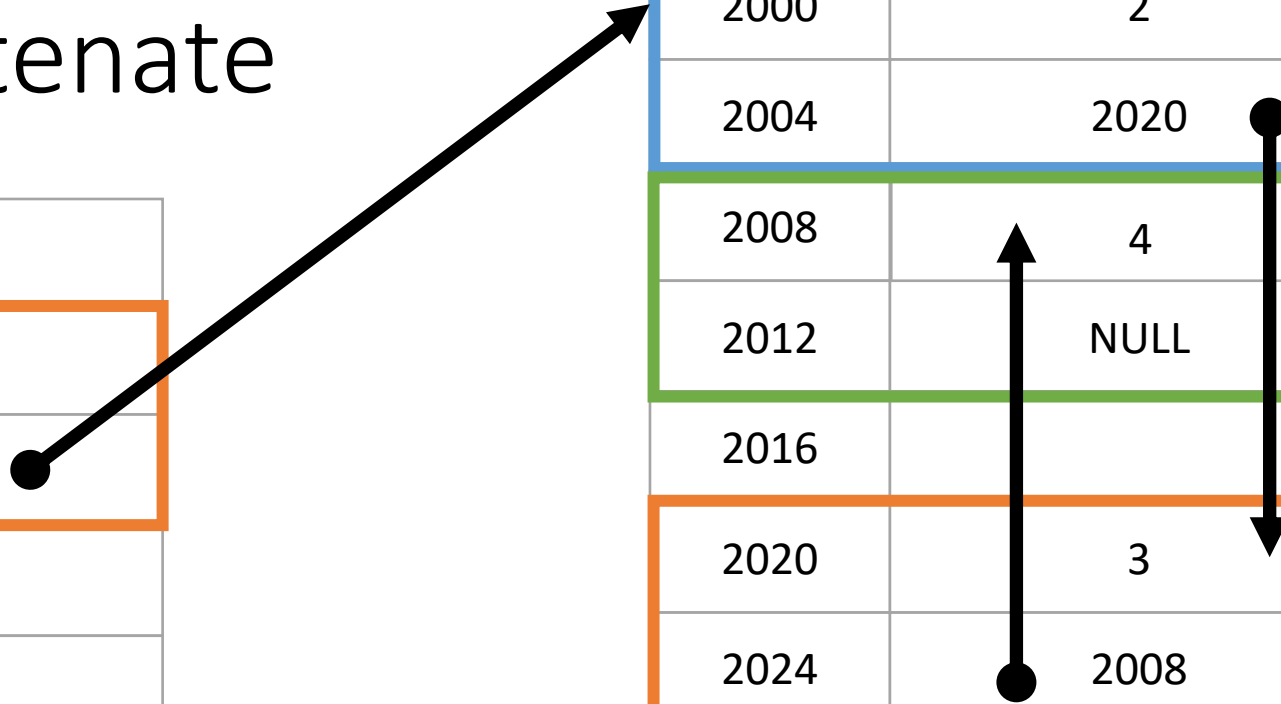
void main(){
    // Testa della lista, nel main
    Node* head = NULL;
}
```

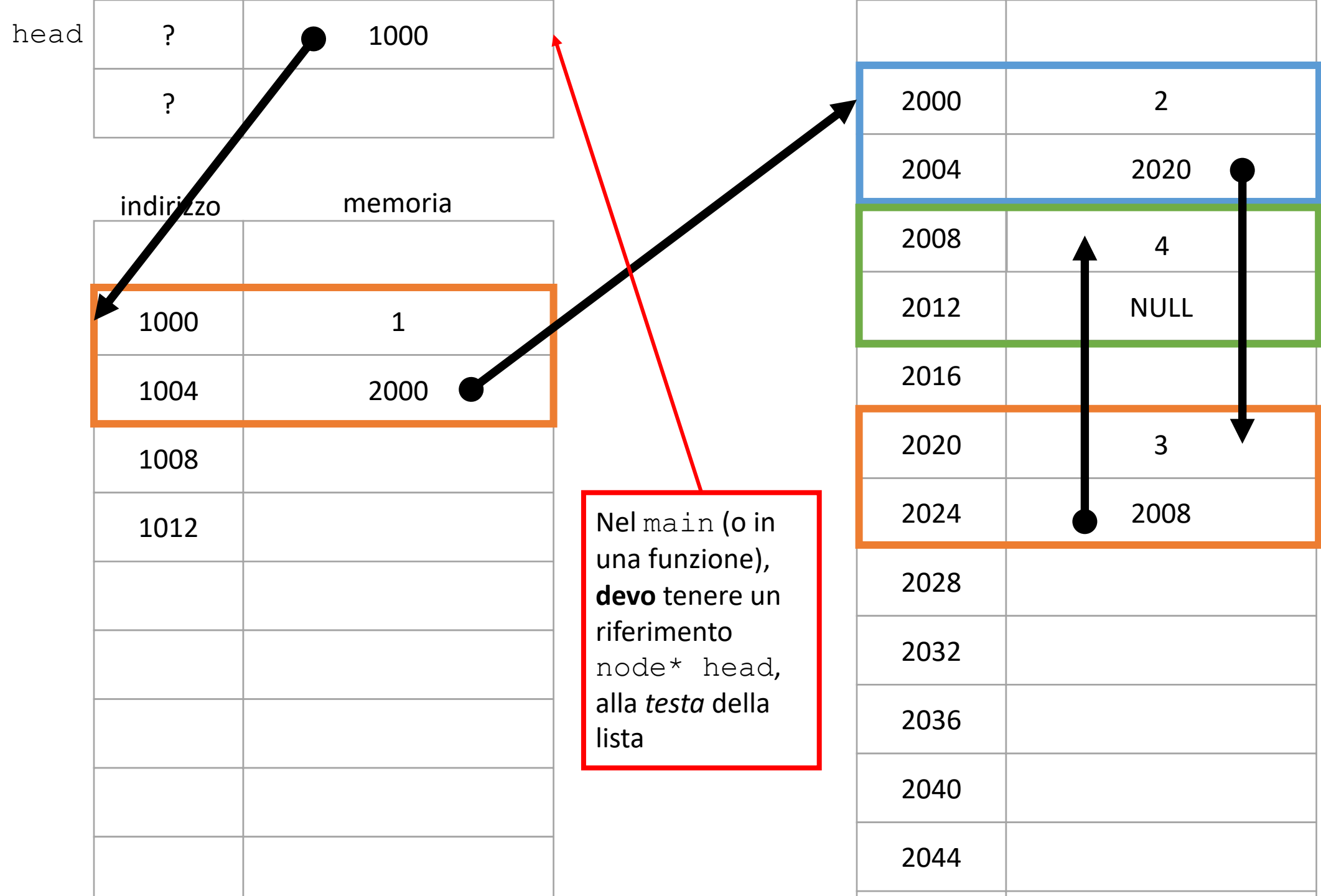


Liste Concatenate

indirizzo	memoria
1000	1
1004	2000
1008	
1012	

2000	2
2004	2020
2008	4
2012	NULL
2016	
2020	3
2024	2008
2028	
2032	
2036	
2040	
2044	





head	?	1000
	?	

indirizzo	memoria
1000	1
1004	2000
1008	
1012	

2000	2
2004	2020
2008	4
2012	NULL
2016	
2020	3
2024	2008
2028	
2032	
2036	
2040	
2044	

Memoria
 allocata
 dinamicamente
 con `malloc`,
 non controllo
 dove viene
 allocata (no
 sequenzialità)

head	?	1000
	?	

indirizzo	memoria
1000	1
1004	2000
1008	
1012	

2000	2
2004	2008
2008	4
2012	NULL
2016	
2020	3
2024	2008
2028	
2032	
2036	
2040	
2044	

Se elimino un elemento, faccio una *free* e devo aggiornare i puntatori degli elementi precedenti/seguenti, ma **non sposto** la loro posizione in memoria

La memoria è libera e può essere riutilizzata, ma non viene inizializzata.

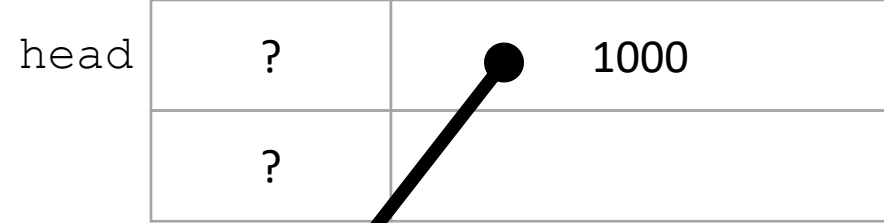
head	?	1000
	?	

indirizzo	memoria
1000	1
1004	2000
1008	
1012	

Attenzione!

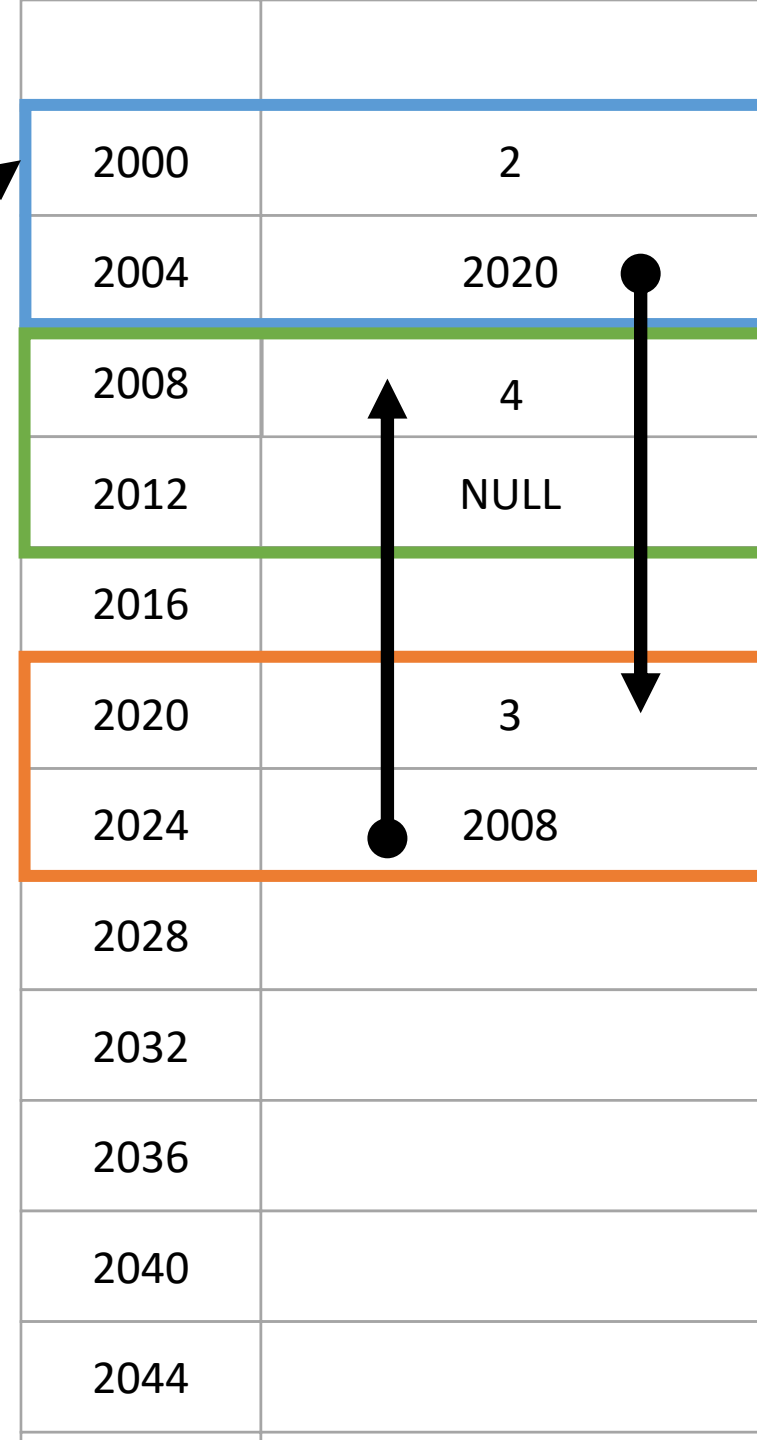
Con una `free` libero la memoria e la rendo disponibile per nuove `malloc`. Se perdo il riferimento alla memoria puntata (aggiorno i puntatori senza fare la `free`), la mia memoria risulterà occupata ma inaccessibile. Questo è un errore (**memory leak**)

2000	2
2004	2008
2008	4
2012	NULL
2016	
2028	
2032	
2036	
2040	
2044	



indirizzo	memoria
1000	1
1004	2000
1008	
1012	

Il campo `*next`
dell'ultimo
elemento punta
a NULL: sono
arrivato alla fine.



Liste Concatenate vs Array Dinamici

```
#include <stdio.h>
#include <stdlib.h>
```

```
typedef struct Node {
    int data;
    struct Node* next;
} Node;
```

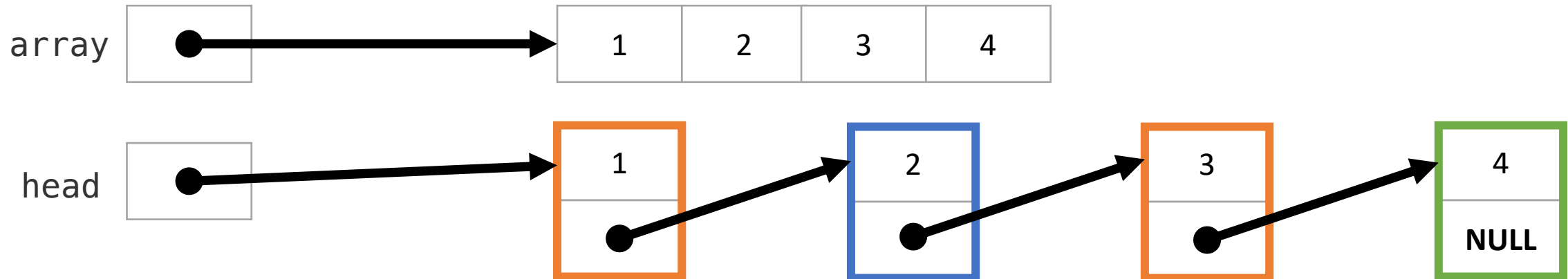
```
void main(){
Node* head = NULL;

}
```

```
int *array;
```

```
...
```

```
array = (int) malloc(lenght*sizeof(int));
```



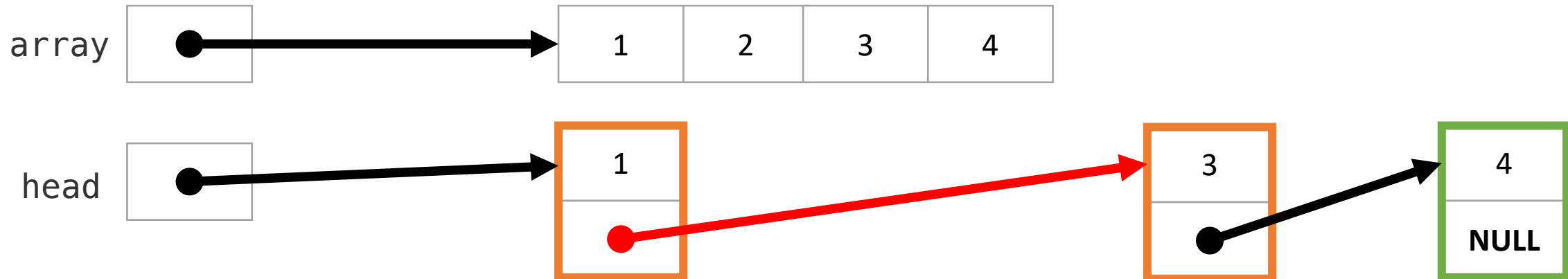
Liste Concatenate vs Array Dinamici

Entrambe possono gestire una struttura dati sequenziali la cui dimensione non è nota a priori.

- + Negli array dinamici gli elementi sono sequenziali, e posso accedere posizionalmente.
- Negli array dinamici cambiare la dimensione (`malloc`, `realloc`) è una operazione costosa.
- Nelle liste concatenate, i dati sono sparsi nella memoria, e non posso accedere sequenzialmente. Devo scorrere la lista.
- + Nelle liste concatenate, aggiungere o togliere un elemento non altera la posizione degli altri elementi in memoria, ed è una operazione poco costosa.

```
typedef struct Node {  
    int data;  
    struct Node* next;  
} Node;  
...  
Node* head = NULL;
```

```
...  
int *array;  
...  
array = (int) malloc(lenght*sizeof(int));
```



Liste Concatenate vs Array Dinamici: Esempio

Array di interi, dimensioni non note a priori; esempio funzioni:

- `init`: inizializza l'array
- `add`: aggiunge un elemento in coda all'array
- `remove`: rimuove un elemento di indice `i`
- `length`: quanti elementi contiene ora
- `free`: cancella l'array
- `get`: restituisce l'`i`-esimo elemento
- `print`: lo stampa a schermo

Versione 1: Array Dinamici

```
typedef struct {  
    int *data; // ptr ai dati  
    int size; // Elementi contenuti  
    int capacity; // max size (dinamica)  
} DynamicArray;
```

```
DynamicArray *initDynamicArray(size_t initialCapacity) {  
    DynamicArray *array = (DynamicArray *)malloc(sizeof(DynamicArray));  
    array->data = (int *)malloc(initialCapacity * sizeof(int));  
    array->size = 0;  
    array->capacity = initialCapacity;  
    return array;  
}
```

```
void addElement(DynamicArray *array, int value) {  
    if (array->size == array->capacity) {  
        resizeArray(array, array->capacity * 2); // Raddoppio la capacità  
    }  
  
    array->data[array->size] = value;  
    array->size++;  
}
```

Dovete sempre controllare che ogni malloc o realloc abbiano successo (non restituiscano NULL).
In questi esempi, ho omesso il controllo per motivi di spazio

Versione 1: Array Dinamici

```
typedef struct {
    int *data; // ptr ai dati
    int size; // Elementi contenuti
    int capacity; // max size (dinamica)
} DynamicArray;

void removeElement(DynamicArray *array, size_t index) {
    if (index >= array->size) {
        printf("Index out of bounds.\n");
        return;
    }
    for (size_t i = index; i < array->size - 1; i++) {
        array->data[i] = array->data[i + 1];
    }
    array->size--;
    if (array->size > 0 && array->size <= array->capacity / 4) {
        resizeArray(array, array->capacity / 2); // riduco capacità
    }
}
```

Attenzione!

Qui la gestione di un indice non valido è semplificata, in una funzione più robusta devo controllare meglio i casi che possono capitare

Versione 1: Array Dinamici

```
typedef struct {  
    int *data; // ptr ai dati  
    int size; // Elementi contenuti  
    int capacity; // max size (dinamica)  
} DynamicArray;
```

```
int getElement(DynamicArray *array, size_t index) {  
    if (index >= array->size) {  
        printf("Index out of bounds.\n");  
        return -1; // Restituisco un valore fuori dal dominio  
    }  
    return array->data[index];  
}  
  
int getLength(DynamicArray *array) {  
    return array->size;  
}
```

Attenzione!

Se -1 è un numero che fa parte del dominio, occorre un'altra variabile per dire se l'operazione si è conclusa in maniera corretta (1) o con errore (-1).

Esempio: `scanf` si comporta in questo modo (passaggio per riferimento)

Versione 2: Linked List

```
typedef struct Node {  
    int data;  
    struct Node *next;  
} Node;
```

```
typedef struct {  
    Node *head;  
    int size;  
} LinkedList;
```

```
LinkedList *initLinkedList() {  
    LinkedList *list = (LinkedList *)malloc(sizeof(LinkedList));  
    list->head = NULL;  
    list->size = 0;  
    return list;  
}
```


Versione 2: Linked List

```
typedef struct Node {
    int data;
    struct Node *next;
} Node;

// Aggiunge un nuovo elemento in coda alla lista
void addElement(LinkedList *list, int value) {
    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = value;
    newNode->next = NULL;

    if (list->head == NULL) {
        list->head = newNode;
    } else {
        Node *current = list->head;
        while (current->next != NULL) {
            current = current->next;
        }
        current->next = newNode;
    }
    list->size++;
}
```

Versione 2: Linked List

```
void removeElement(LinkedList *list, size_t index) {  
    if (index >= list->size) {  
        printf("Index out of bounds.\n");  
        return;  
    }  
  
    Node *current = list->head;  
    Node *previous = NULL;  
  
    if (index == 0) {  
        list->head = current->next;  
        free(current);  
    } else {  
        for (size_t i = 0; i < index; i++) {  
            previous = current;  
            current = current->next;  
        }  
        previous->next = current->next;  
        free(current);  
    }  
    list->size--;  
}
```

Versione 2: Linked List

```
typedef struct Node {
    int data;
    struct Node *next;
} Node;

int getElement(LinkedList *list, size_t index) {
    if (index >= list->size) {
        printf("Index out of bounds.\n");
        return -1; // Come prima; -1 va bene come valore se non e' nel dominio
    }

    Node *current = list->head;
    for (size_t i = 0; i < index; i++) {
        current = current->next;
    }
    return current->data;
}
```

```
int increase(int* a){  
    *a = *a+1;  
    return *a;  
}
```

```
int main(){  
    int x = 0;  
    printf("%d %d %d\n", x, increase(&x), x);  
    return 0;  
}
```

Cosa stampa?

// che cosa stampo?

```
int x=1;
```

```
x = x++;
```

```
printf("%d\n", x);
```

Liste Concatenate: Errori Tipici

- Non gestire il caso lista-vuota; aggiungere il primo elemento è spesso una operazione che va gestita con un caso specifico.
- Non gestire il caso di lista che *ritorna* ad essere vuota: cosa succede se tolgo *tutti* gli elementi?
- Gestire in maniera errata la fine della lista, ad esempio arrivare a un elemento

`current == NULL`

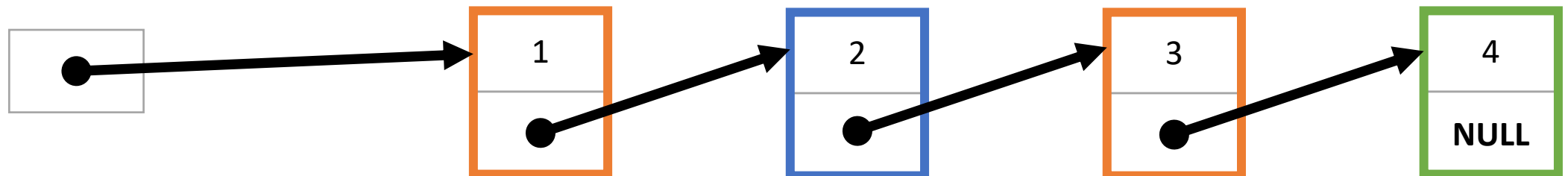
e fare

`current->next`

ovvero accedere al campo `next` di `null`

- Perdere la testa della lista mentre la si scorre
- Omettere la chiamata a `free` quando viene eliminato un elemento

In generale, dovete prestare particolare attenzione ai casi limiti di lista vuota e «fine» della lista.



Undefined Behaviour

Comportamento *non consentito*, ma che il compilatore *non* riesce a rilevare.

- Un (nuovo?) tipo di errore: il programma fa qualcosa, ma non ho controllo sul risultato
- Spesso causa **warning**, ma dipende da parametri di compilazione e dal tipo di errori. Molti sono silenti
- Spesso dovuti a un compromesso tra velocità di esecuzione del codice e sicurezza; la maggior parte dei comportamenti indefiniti *potrebbero* essere rilevati con dei controlli che contano qualche ciclo di CPU
- Esempio: index out-of-bounds di un array, usare un dato non inizializzato
- Linguaggi più astratti → + controlli, meno undefined behaviour, + lenti
- In C: sta a voi evitare di creare undefined behavior

Undefined Behaviour

```
int increase(int* a){  
    *a = *a+1;  
    return *a;  
}
```

```
int x=1;  
x = x++;  
printf("%d\n", x);
```

```
int main(){  
    int x = 0;  
    printf("%d %d %d\n", x, increase(&x), x);  
    return 0;  
}
```

Cosa stampa? Non importa, **sono dati errati in ogni caso.**

- L'ordine di valutazione degli argomenti non è valutato dallo standard
- Su Windows/Unix risultato è diverso (dx vs sx)
- In entrambi i casi, è un errore

Undefined Behaviour

```
char *p = "wikipedia"; // valido in C,  
                        // deprecato C++  
p[0] = 'W'; // undefined behavior  
printf("%s\n", p);
```

Modificare uno string literal è un undefined behaviour

Uno string literal è una stringa anonima definita tra apici nel codice sorgente (esempio, un nome di un file hardcodato).

Undefined Behaviour

Dividere per 0 è undefined behaviour

```
int x = 1;  
return x / 0;
```

Index out of bounds, anche

```
int arr[4] = {0, 1, 2, 3};  
int *p = arr + 5; // index out of bounds
```

Dereferenziare un NULL pointer, anche

```
p = NULL;  
int a = *p; // dereferenziazione null pointer
```

Accedere ad una variabile non inizializzata, anche

```
int x, y;  
y = x + 1; // variabile non inizializzata
```

Undefined Behaviour

Comparare due puntatori appartenenti a due oggetti diversi è UB

```
int a = 0; // comparazione
int b = 0; // due pointer su oggetti diversi
return &a < &b; /* undefined behavior */
```

Anche fare modifiche non sequenziali a una stessa variabile

```
int ud5(int i) {
    return i++ + i++; /* 2 modifiche non
sequenziali a i*/
}
```

Undefined Behaviour

```
int* ud6() { // -Ox -> diversi livelli di ottimizzazione compiler
    int x = 1; // provate a compilarlo con gcc -O1 (stampa 1)
    return &x; // e gcc -O2 (stampa random)
}

int ud6Main() {
    printf("%d\n", *ud6());
    return 0;
}
```

Qui stiamo provando ad accedere ad un elemento che sta nel record di attivazione di una funzione, dopo la terminazione della stessa.
Il comportamento è *undefined* e cambia a seconda di livelli diversi di ottimizzazione del compilatore.

Undefined Behaviour

```
double d;  
scanf("%d",&d);  
printf("%d\n",d); // UB dovuto a va_args  
printf("%d\n",d); //
```

Qui il tipo di dato e il modo in cui lo leggo (o scrivo) non corrispondono.

Quando ci accedo, il risultato dipende a seconda di come C accede alle variabili passate a funzioni con parametri multipli (come `printf` e `scanf`).

Anche qui, a prescindere da cosa succede, è un errore.

Undefined Behaviour

- Difficili da trovare e da debuggare
- Spesso non causano warning né errori a runtime
- Spesso il risultato “undefined” è quello che voglio, e quindi fa sembrare il codice “corretto”

Come evitarli?

- Conoscere lo standard C
- Seguire le buone pratiche di programmazione
- Fare attenzione ai warning
- Parametri del compilatore

Undefined Behaviour

- Difficili da trovare e da debuggare
- Spesso non causano warning ne errori a runtime
- Spesso il risultato “undefined” è quello che voglio, e quindi fa sembrare il codice “corretto”

Come evitarli?

- Conoscere lo standard C
- Seguire le buone pratiche di programmazione
- Fare attenzione ai warning
- Parametri del compilatore

Se scrivete codice semplice, è difficile fare undefined behaviors.

Scrivere codice più complesso richiede una conoscenza più profonda delle specifiche dello standard C per evitare questo tipo di errori.

Undefined Behaviour

- An example of ***unspecified behavior*** is the order in which the arguments to a function are evaluated.
- An example of ***undefined behavior*** is the behavior on integer overflow.
- An example of ***implementation-defined behavior*** is the propagation of the high-order bit when a signed integer is shifted right.